University of Ottawa

CSI 3140 - WWW Structures, Techniques and Standards

Laboratory 3

JavaScript, DOM Manipulation, Events, Validation, and Dynamic Page Behavior

Professor	Mohamed Ali Ibrahim, PhD, PEng
Term	Spring/Summer 2026
Lab Period	Monday, June 15, 2026 to Saturday, June 27, 2026
Submission	Brightspace, by Saturday, June 27, 2026, 11:59 PM
Team Size	2-3 students
Technologies	HTML5, CSS3, JavaScript, DOM API, events, forms, browser developer tools, HTML/CSS/JavaScript validation tools

1. Laboratory Overview

The purpose of this laboratory is to give students hands-on practice with client-side JavaScript, DOM manipulation, event handling, form validation, and dynamic page behavior. Students will extend a multi-page website by adding interactive features that respond to user actions and modify the content or presentation of the page dynamically.

This laboratory is aligned with the course topics related to JavaScript fundamentals, variables, functions, conditional statements, loops, arrays, objects, DOM selection and manipulation, event listeners, forms, validation, error handling, debugging, and accessibility-aware interactive Web design.

Students may continue the website developed in Laboratory 2. However, the main grading focus of this laboratory is now on JavaScript behavior, DOM interaction, event-driven programming, validation, and dynamic user experience.

2. Topics Covered in Lab 3

Topic Area	Required Coverage
JavaScript foundations	Variables, constants, functions, expressions, conditional statements, loops, arrays, objects, comments, and clear code organization.
DOM manipulation	Selecting elements, changing text content, modifying attributes, adding or removing classes, creating or removing elements, and updating page content dynamically.
Event handling	Click events, input events, change events, submit events, keyboard events where appropriate, and event listener organization.
Dynamic page behavior	Interactive menus, buttons, filters, galleries, accordions, tabs, counters, calculators, previews, or other meaningful dynamic features.
Form validation	Required fields, input format checking, error messages, visual feedback, prevention of invalid submission, and user-friendly validation behavior.
Debugging and testing	Browser developer tools, console messages, JavaScript error checking, validation tools, and testing across pages and browsers.
Accessibility-aware interaction	Keyboard usability, visible focus states, readable error messages, non-color-only feedback, and logical interaction

	flow.

3. Learning Objectives

- Write JavaScript code in an external .js file and connect it correctly to HTML pages.
- Use JavaScript variables, functions, conditionals, loops, arrays, and objects to implement page logic.
- Select and manipulate DOM elements using JavaScript.
- Respond to user actions using event listeners.
- Modify page content, classes, attributes, and styles dynamically.
- Implement client-side form validation with clear error messages.
- Use browser developer tools to debug JavaScript errors and inspect DOM changes.
- Create interactive features that improve the usability of a website.
- Test and document JavaScript behavior in a short technical report.

4. Problem Description

Each team must extend or create a multi-page website for a fictional university event, club, workshop, technical mini-course, academic service, student association, or similar topic. The website must demonstrate meaningful JavaScript functionality.

Teams are encouraged to continue the website developed in Laboratory 2, but they must now add JavaScript-driven interactivity. The JavaScript must not be decorative only. It must provide real dynamic behavior that improves how users interact with the website.

The website must use valid HTML5, CSS3, and JavaScript. JavaScript code must be placed in one or more external .js files. Inline JavaScript should be avoided unless explicitly justified in the report.

5. Required Website Structure

File	Required Content
index.html	Home page introducing the website topic. Must include at least one JavaScript-powered interactive section, such as a welcome message, dynamic announcement, featured item, counter, or interactive button.
about.html	Information page explaining the event, club, workshop, or topic. Must include at least one DOM-based interactive feature, such as expandable sections, tabs, show/hide content, or dynamically updated information.
schedule.html	Schedule, program, services, or activity page. Must include JavaScript interaction such as filtering, sorting, highlighting, category selection, or dynamic display of schedule items.
contact.html or registration.html	Contact or registration page. Must include a form with client-side JavaScript validation and clear error messages.
styles.css	External stylesheet used consistently across all pages. It may reuse the Lab 2 stylesheet and must support the visual design of the interactive elements.
script.js	Main external JavaScript file used to implement DOM manipulation, event handling, validation, and dynamic page behavior. Additional JavaScript files may be used only if the structure is clearly explained in the report.

All pages must include consistent navigation and must be connected using relative hyperlinks. Each page must include the viewport meta tag and must remain usable at different screen sizes.

6. Part A - JavaScript Organization and External Script File

All JavaScript must be placed in one or more external .js files. Inline JavaScript should not be used, except for rare cases that are explicitly justified in the report. The JavaScript file should be organized with comments and logical sections.

- Use at least one shared external JavaScript file linked from the relevant HTML pages.
- Use meaningful variable and function names.
- Organize JavaScript code using functions.
- Avoid unnecessary repetition by creating reusable functions.
- Include comments to explain major parts of the JavaScript code.
- Ensure that JavaScript code does not run before the DOM elements are available.

```
<script src="js/script.js" defer></script>
```

```
function updateWelcomeMessage() {
  const message = document.querySelector("#welcome-message");
  message.textContent = "Welcome to our interactive CSI 3140 website!";
}
```

7. Part B - DOM Selection and Manipulation

The website must demonstrate clear use of the DOM API. Students must select HTML elements and modify them dynamically using JavaScript.

- Select elements using methods such as querySelector, querySelectorAll, getElementById, or equivalent DOM methods.
- Change text content or HTML content dynamically.
- Add, remove, or toggle CSS classes.
- Modify attributes when appropriate.
- Create, insert, or remove elements dynamically in at least one meaningful section.
- Avoid unnecessary direct inline styling in JavaScript when a CSS class can be used instead.

```
const button = document.querySelector("#theme-button");
const page = document.querySelector("body");

button.addEventListener("click", function () {
  page.classList.toggle("dark-mode");
});
```

8. Part C - Event Handling

The website must respond to user actions using event listeners. Students must use JavaScript events to make the website interactive.

- Use addEventListener instead of inline event attributes such as onclick.
- Include at least three different event-driven interactions.
- Use click events for buttons, menus, cards, tabs, or other interactive components.
- Use input, change, or submit events for form-related behavior.
- Provide visual or textual feedback after user actions.
- Ensure that interactive behavior is understandable and predictable.

Examples of acceptable event-driven features include:

- A navigation menu that opens and closes on smaller screens.

- A button that shows or hides additional information.
- A set of tabs that displays different content.
- A schedule filter that displays selected activities.
- A form that validates fields before submission.
- A dynamic preview that updates as the user types.

9. Part D - Dynamic Page Behavior

Students must implement meaningful dynamic page behavior. The goal is to demonstrate that JavaScript can change the page after it has loaded.

Each team must implement at least three meaningful dynamic features, selected from the following list or approved equivalents:

- Show/hide content section.
- Accordion or collapsible FAQ.
- Tabbed content area.
- Image gallery or carousel.
- Schedule filter by category, day, or type.
- Dynamic list creation from an array of objects.
- Character counter or live input preview.
- Simple calculator or cost estimator.
- Theme toggle or display preference option.
- Dynamic announcement or message based on user input.
- Interactive quiz or knowledge check.
- Modal window or pop-up information box.

At least one feature must involve creating or updating content from a JavaScript array or object.

```
const sessions = [
  { title: "HTML Review", type: "Workshop" },
  { title: "CSS Layout Practice", type: "Lab" },
  { title: "JavaScript DOM Activity", type: "Tutorial" }
];

function displaySessions() {
  const list = document.querySelector("#session-list");
  list.innerHTML = "";

  sessions.forEach(function (session) {
    const item = document.createElement("li");
    item.textContent = `${session.title} - ${session.type}`;
    list.appendChild(item);
  });
}
```

10. Part E - Form Validation

The contact or registration page must include a form with client-side JavaScript validation. The form must provide clear feedback before submission.

The form must include at least four input fields, such as:

- Full name.

- Email address.
- Student number or username.
- Program or year of study.
- Event/session selection.
- Message or comment.
- Checkbox confirmation.
- Radio buttons or dropdown list.

Students must:

- Validate that required fields are not empty.
- Validate the email format.
- Validate at least one additional field using a rule chosen by the team.
- Display clear error messages beside or near the relevant fields.
- Prevent submission when the form contains invalid data.
- Provide a success message or confirmation when the form is valid.
- Use accessible labels and meaningful error text.

```
form.addEventListener("submit", function (event) {
  event.preventDefault();

  const email = document.querySelector("#email");
  const error = document.querySelector("#email-error");

  if (!email.value.includes("@")) {
    error.textContent = "Please enter a valid email address.";
    email.classList.add("input-error");
  } else {
    error.textContent = "";
    email.classList.remove("input-error");
  }
});
```

11. Part F - JavaScript Data Structures

Students must use at least one JavaScript array or object to store and display information dynamically.

Examples include:

- A list of event sessions.
- A list of club members.
- A list of services.
- A list of announcements.
- A list of frequently asked questions.
- A list of registration options.
- A list of project resources.

Students must:

- Define an array or object in JavaScript.
- Use a loop such as for, for...of, or forEach.
- Dynamically display the information on the page.
- Explain in the report why this data structure was used.

```
const announcements = [
  "Registration opens on June 15.",
  "Workshop seats are limited.",
  "Please bring your laptop to the lab session."
];

announcements.forEach(function (announcement) {
  const item = document.createElement("li");
  item.textContent = announcement;
  document.querySelector("#announcements").appendChild(item);
});
```

12. Part G - Accessibility-Aware Interaction

JavaScript must improve the user experience without reducing accessibility. Interactive features must remain understandable and usable.

- Ensure that buttons and interactive elements can be reached using the keyboard.
- Avoid using only color to indicate errors or state changes.
- Provide readable text feedback for validation errors.
- Use appropriate labels for form controls.
- Keep focus styles visible.
- Ensure that hidden or displayed content changes do not confuse users.
- Ensure that dynamic content remains logically placed in the page.

For example, when displaying validation errors, the message should clearly explain the problem:

```
<span id="email-error" class="error-message"></span>
```

13. JavaScript and Dynamic Behavior Checklist

Each team must complete the following checklist and include it in the report.

Item	Completed?
All relevant pages link correctly to the external JavaScript file.	
JavaScript code is placed in an external .js file.	
Inline JavaScript is avoided or explicitly justified.	
The JavaScript file is organized with meaningful comments and functions.	
DOM elements are selected and modified dynamically.	
At least three meaningful event-driven interactions are implemented.	
At least one dynamic feature uses an array or object.	
The website includes form validation using JavaScript.	
Clear error messages are displayed for invalid form input.	
Invalid form submission is prevented.	
A success or confirmation message is displayed when the form is valid.	
Interactive elements are usable with the keyboard.	
JavaScript errors were checked using browser developer tools.	
The website was tested in desktop, tablet, and mobile widths.	
HTML, CSS, and JavaScript files were checked using appropriate validation or debugging tools.	

14. Laboratory Demonstration Requirement

Each team must demonstrate the laboratory work to the TA. The report will be considered for grading only if the demonstration has been completed. All team members must be present during the demonstration.

During the demonstration, the TA may ask any team member to explain the JavaScript organization, DOM manipulation, event listeners, dynamic features, form validation rules, testing process, debugging steps, and the role of each submitted file. Each student must be able to explain the team submission clearly.

15. Testing Requirements

Each team must test the website before submission. The testing must include:

- Opening each HTML page in a modern browser.
- Checking that all internal navigation links work correctly.
- Checking that the external CSS file loads correctly on every page.
- Checking that the external JavaScript file loads correctly on the relevant pages.
- Testing all buttons, menus, tabs, filters, forms, and other interactive elements.
- Testing the form with valid and invalid input.
- Confirming that invalid form submission is prevented.
- Checking the browser console for JavaScript errors.
- Using browser developer tools to inspect DOM updates.
- Testing the website at desktop, tablet, and mobile widths.
- Checking keyboard navigation and visible focus states.
- Validating HTML and CSS using appropriate validation tools.
- Reviewing JavaScript code for syntax errors and logical errors.

16. Deliverables

Each team must submit one ZIP file on Brightspace containing:

- All HTML files used for the website.
- All CSS files used for styling.
- All JavaScript files used for interactivity.
- Any image or media files used in the website.
- A short lab report in PDF format.
- Screenshots showing the website pages in a browser.
- Screenshots showing the interactive features before and after user actions.
- Screenshots showing form validation with invalid and valid input.
- Screenshots showing desktop, tablet, and mobile views.
- Screenshots or exported results showing validation or debugging evidence.
- Optional: a short README file explaining how to open and test the website.

17. Report Requirements

The lab report must include:

- Course code, lab number, team members, and student numbers.
- A short description of the website topic and the interaction goals.
- A file structure diagram or list explaining all submitted files.

- A description of the JavaScript file organization.
- A description of the DOM elements selected and modified.
- A description of each event-driven feature implemented.
- An explanation of the dynamic feature that uses an array or object.
- A description of the form validation rules.
- Screenshots of validation errors and successful validation.
- A description of the testing and debugging process.
- The completed JavaScript and dynamic behavior checklist.
- Screenshots of desktop, tablet, and mobile views.
- A short reflection on problems encountered and how they were solved.

18. Grading Rubric

Component	Points
Correct multi-page website structure, file organization, and external CSS/JavaScript linking	10
JavaScript organization, readability, comments, reusable functions, and maintainable code structure	10
Correct DOM selection and manipulation, including updating content, classes, attributes, or elements	15
Meaningful event handling using addEventListener and at least three user interactions	15
Dynamic page behavior, including at least one feature based on an array or object	15
Form validation: required fields, email validation, custom rule, error messages, and submission control	15
Accessibility-aware interaction: keyboard usability, readable feedback, labels, and focus visibility	5
Testing, debugging, browser console checks, screenshots, and evidence of corrections	5
Laboratory demonstration to the TA with all team members present	3
Report quality, clarity, screenshots, checklist, and reflection	7
Total	**100**

19. Important Notes

- Late submissions will not be accepted unless an official accommodation or approved exceptional circumstance applies.
- All team members must understand the submitted website, JavaScript code, and report.
- External code, templates, images, fonts, icons, or resources must be cited if used and must be permitted for academic use.
- Students must not submit a website generated entirely by an external tool without understanding, adapting, and documenting the work.
- Academic integrity rules apply to all submitted files and reports.
- The use of JavaScript libraries or frameworks such as React, Angular, Vue, jQuery, Bootstrap JavaScript components, or similar tools is not permitted for this laboratory unless explicitly approved by the instructor. The objective is to practice writing JavaScript manually.
- Browser alerts may be used for simple testing, but final validation feedback should be displayed clearly on the page using DOM manipulation.

20. Timeline

Date	Suggested Work
Monday, June 15	Read the lab statement, confirm the website topic, review the Lab 2 structure if reused, and plan the JavaScript features.
June 16-17	Create or clean the HTML/CSS/JavaScript file structure and link the external JavaScript file to the relevant pages.
June 18-19	Implement DOM selection and manipulation features, including show/hide sections, tabs, dynamic content, or other interactions.
June 20-21	Implement event listeners and at least three meaningful event-driven interactions.
June 22-23	Implement the form validation rules, error messages, and successful submission feedback.
June 24	Implement or finalize the dynamic feature using an array or object.
June 25	Test all JavaScript features, check the browser console, debug errors, and test responsive behavior.
June 26	Complete the report, checklist, screenshots, validation evidence, and final testing.
Saturday, June 27	Submit the ZIP file on Brightspace by 11:59 PM.